Moteur Physique : Détection et Réponse aux Collisions

Rapport de projet – Physique Appliquée

Matéo Baron

30 octobre 2026

1 Introduction et architecture

Ce projet implémente un moteur physique de collision entièrement « fait maison » sur Unity (C#), sans aucun `Rigidbody`. Il comprend une broad phase (Dynamic AABB Tree), une narrow phase (GJK + EPA) et une réponse avec rotation. La scène de démonstration est un plateau de type Plinko (Figure 2) : des billes tombent à travers des rangées de piquets et le joueur peut appuyer sur C ou S pour faire tomber des capsules ou des sphères. Cette scène sollicite l'ensemble du moteur : la broad phase filtre les paires bille-piquet et bille-bille, GJK et EPA fournissent les informations de contact, et la réponse gère les rebonds et la rotation des billes.

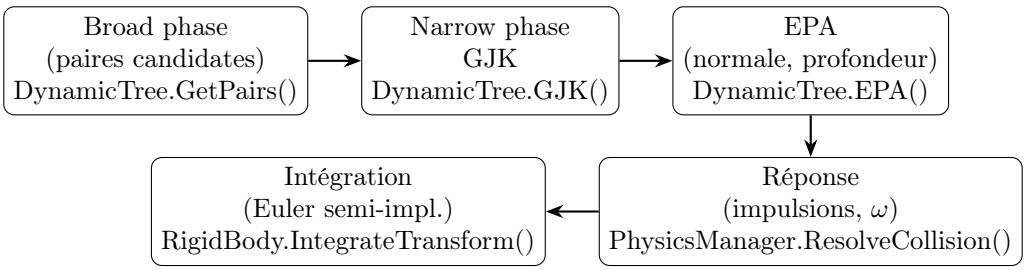


FIGURE 1 – Pipeline exécuté à chaque `FixedUpdate`.

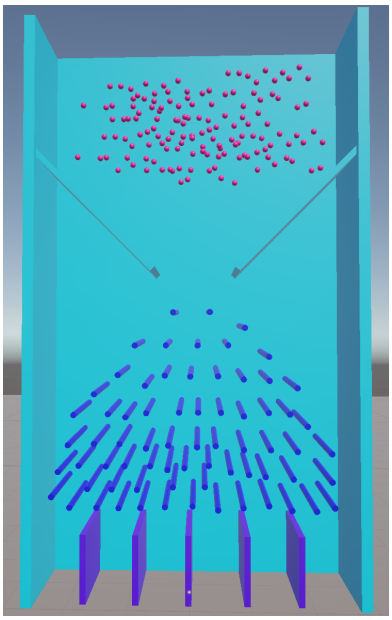


FIGURE 2 – Scène de démonstration (plateau de type Plinko).

2 Broad phase

Choix : j'ai implémenté un Dynamic AABB Tree, car la scène mélange des objets statiques (piquets) et dynamiques (billes) et que sa taille varie selon le plateau testé. Contrairement à une grille, l'arbre ne nécessite pas de régler une taille de cellule, et contrairement au Sweep and Prune, ses performances ne dépendent pas d'un axe de projection privilégié.

Implémentation : chaque objet est une feuille de l'arbre, avec une AABB agrandie d'une marge réglable dans l'éditeur. Les paires candidates sont obtenues par `DynamicTree.GetPairs()`. Un objet

n'est réinséré que lorsqu'il sort de son AABB agrandie.

Insertion / mise à jour / suppression :

- `InsertNode()` agrandit l'AABB (`Fatten()`), descend l'arbre en choisissant à chaque étape l'enfant dont l'aire augmenterait le moins (coût de surface), puis crée la feuille et un nœud parent interne avant de remonter les bornes (`UpdateParentBounds()`).
- `UpdateTree()` réinsère paresseusement un collider dont l'AABB n'est plus contenue dans celle de sa feuille agrandie (`RemoveLeaf()` puis `InsertNode()`).
- `RemoveLeaf()` libère la feuille, remplace son parent par le frère et recalcule les bornes des ancêtres. `RemoveCollider()` enchaîne `RemoveLeaf()` et un `RemoveAtSwapBack()` sur `colliders`.

Difficulté — objets statiques : les piquets ne bougent jamais, leurs AABB ne sont donc jamais invalidées. Le filtrage est délégué à `CrossChildren()` : une paire n'est ajoutée que si au moins un des deux corps est dynamique (`mass != 0`), évitant les paires statique/statique inutiles. Le test `Collide` reste toutefois effectué sur ces paires avant filtrage : surcoût minime, évitable en marquant les sous-arbres purement statiques.

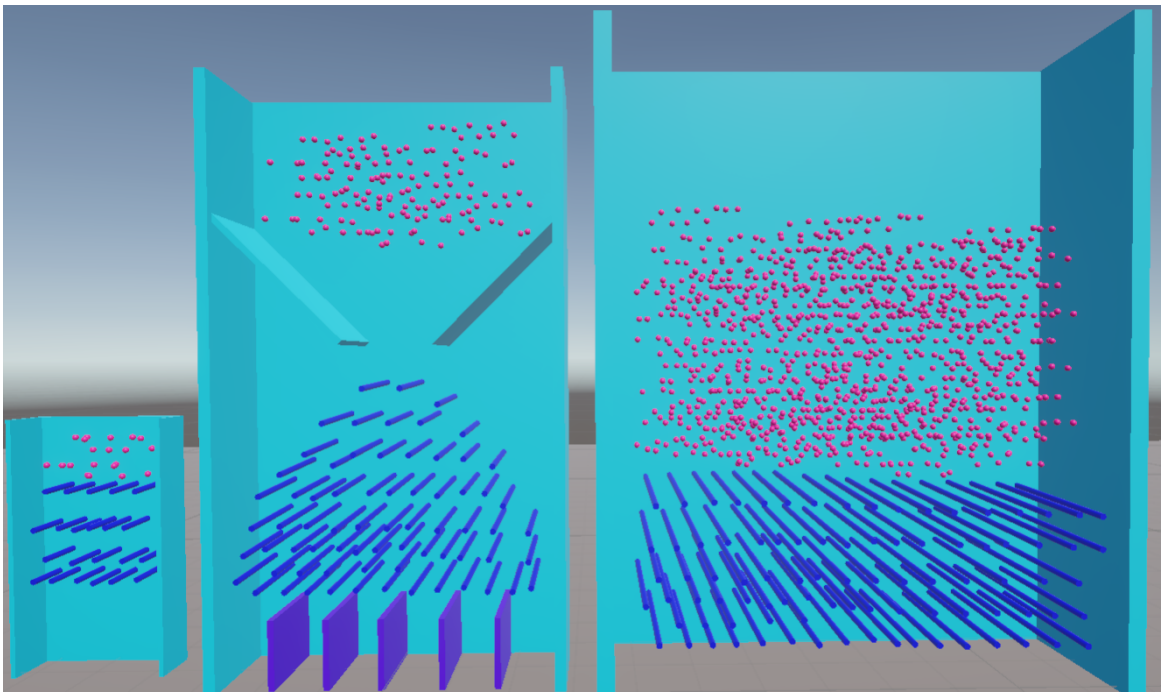


FIGURE 3 – Plateaux utilisés pour le banc d'essai (50, 200 et 1000 objets).

Résultats : Le tableau 1 compare l'approche naïve en $O(n^2)$ à ma broad phase.

n objets	Paires naïves	Paires candidates	Temps (ms)
50	1 225	20	0,04
200	19 900	160	0,19
1 000	499 500	1500	0,74

TABLE 1 – Réduction du nombre de paires testées :

À 1 000 objets, seules 0,30 % des paires naïves sont conservées. Le temps de la broad phase reste inférieur à 1 ms. Le nombre de paires candidates augmente nettement lorsque les billes atteignent le sol, car elles s'y accumulent et leurs AABB se recouvrent.

Marge	Paires/fixedUpdate	Réinsertions/fixedUpdate	Temps total (ms)
0	60	140	5,3
0,2	160	50	4,7
0,5	280	24	5,2
1	590	11	6,2

TABLE 2 – Influence de la marge de l'AABB agrandie.

Influence de la marge : J'ai comparé plusieurs marges sur la même scène ($n = 200$, 10 s de simulation, tableau 2). Une marge de 0 provoque beaucoup de réinsertions (140 par `fixedUpdate`), alors qu'une marge trop grande (1) génère trop de paires candidates (590) et donc des faux positifs pour la narrow phase. Le temps total est minimal pour 0,2 (4,7 ms), valeur que j'ai retenue.

3 Narrow phase : GJK et EPA

GJK : Chaque paire candidate est testée par `DynamicTree.GJK()`, qui s'appuie sur la fonction support (BaseCollider.Support()) de chaque collider (sphère, capsule, cube, mesh), ou indique l'absence de collision. Le nombre d'itérations est limité à 64 pour éviter toute boucle infinie.

EPA : Lorsque GJK détecte une collision, `DynamicTree.EPA()` étend le polytope pour en déduire la normale de contact, la profondeur de pénétration et le point de contact (obtenu par coordonnées barycentriques sur la face la plus proche).

3.1 Debug visuel

Un mode de debug, activable dans l'éditeur sur les colliders et le DynamicTree, permet de visualiser le fonctionnement de la détection de collision :

- **Bounds des colliders :** l'AABB de chaque collider est dessinée.
- **Bounds de l'arbre :** l'AABB agrandie de chaque feuille ainsi que celle de tous les nœuds internes (parents) sont affichées, ce qui permet de vérifier la construction et l'évolution du Dynamic AABB Tree.
- **Paires testées sans collision :** une ligne verte est tracée entre les deux colliders lorsqu'une paire est testée par GJK mais qu'aucune collision n'est détectée.
- **Collisions :** lorsqu'une collision est détectée, une sphère et une ligne rouge sont dessinée au point de contact représentant la normale et la profondeur de pénétration calculée par EPA.

4 Réponse aux collisions

Rigidbody : Chaque objet possède une masse (masse nulle = objet statique), un coefficient de restitution (`Bounciness`), une vitesse linéaire et une vitesse angulaire. L'intégration est faite dans `RigidBody.IntegrateTransform()`.

Tenseurs d'inertie : l'inertie locale est stockée sous forme de la diagonale de la matrice en local space (`invLocalInertia`) dans le repère de l'objet, puis transformée en world space via `ApplyInvInertia()` (rotation inverse, mise à l'échelle composante par composante, rotation directe). Pour une **sphère** de masse m et de rayon r, le tenseur est :

$$I_{xx} = I_{yy} = I_{zz} = \tfrac{2}{5}\, mr^2. \tag{1}$$

Pour une **capsule** de rayon r, de hauteur de segment h (partie cylindrique) et de masse m, on combine l'inertie du cylindre et celle des deux demi-sphères (théorème des axes parallèles) :

$$I_{yy} = \tfrac{1}{2} m_c r^2 + \tfrac{4}{5} m_s r^2, \qquad I_{xx} = I_{zz} = m_c\Big(\tfrac{h^2}{12} + \tfrac{r^2}{4}\Big) + m_s\Big(\tfrac{h^2}{4} + \tfrac{3hr}{8} + \tfrac{2}{5} r^2\Big), \tag{2}$$

avec m_c et m_s les masses respectives du cylindre et des demi-sphères. Le calcul est effectué une seule fois au démarrage à partir de la géométrie du collider.

Impulsion : `PhysicsManager.ResolveCollision()` reçoit les informations de contact fournies par EPA et applique une impulsion le long de la normale, seulement si les objets se rapprochent. Son intensité est donnée par :

$$j = \frac{-(1+e)\,(\mathbf{v}_{rel} \cdot \mathbf{n})}{\frac{1}{m_1} + \frac{1}{m_2} + (\mathbf{r}_1 \times \mathbf{n}) \cdot I_1^{-1}(\mathbf{r}_1 \times \mathbf{n}) + (\mathbf{r}_2 \times \mathbf{n}) \cdot I_2^{-1}(\mathbf{r}_2 \times \mathbf{n})}, \qquad e = \frac{e_1 + e_2}{2} \tag{3}$$

Cette impulsion modifie à la fois les vitesses linéaires et angulaires, ce qui fait tourner les billes après un choc avec un piquet.

Correction de position. Pour éviter l'enfoncement, les objets sont écartés le long de la normale, proportionnellement à leur masse inverse : un objet statique ne bouge pas et l'autre absorbe toute la correction.

5 Résultats et analyse

n objets	FPS	Update arbre (ms)	Broad (ms)	Narrow (ms)
50	500	0,16 [0,03–1,00]	0,04 [0,01–0,16]	0,55 [0,02–3,74]
200	410	1,29 [0,11–8,08]	0,19 [0,08–0,50]	2,47 [0,10–9,69]
1 000	9	7,75 [0,57–17,59]	0,74 [0,36–1,13]	11,85 [1,00–30,87]

TABLE 3 – Moyenne [min–max] par frame, build Release.

Variation au cours de la simulation : À $n = 1000$, le nombre de paires candidates passe de 634 à 3 312 (moyenne 1 500) et le temps de la narrow phase de 1,0 à 30,9 ms (moyenne 11,8 ms). Ces deux valeurs évoluent ensemble : les billes s'accumulent au sol, leurs AABB se recouvrent, et davantage de paires aboutissent à un contact complet GJK+EPA. La broad phase, elle, varie peu (0,36 à 1,13 ms).

FPS à 1 000 objets : Les trois phases mesurées totalisent environ 20 ms en moyenne, ce qui ne correspond pas aux 9 FPS observés. L'explication tient à la dynamique de la scène : au fur et à mesure que les billes tombent et s'accumulent, elles se rapprochent, leurs AABB se recouvrent de plus en plus, et le nombre de paires candidates augmente. La narrow phase atteint alors des pics à 30,9 ms, soit *plus que le pas de temps physique* (`Time.fixedDeltaTime` = 20 ms par défaut). Le moteur ne peut donc plus tenir le rythme : chaque `FixedUpdate` dépasse son budget, les pas de simulation s'accumulent et le jeu ralentit. Le coût des deux autres phases (update arbre et broad phase) reste marginal en comparaison.

Méthodologie de mesure : J'ai mesuré `PhysicsManager.FixedUpdate()` avec un `Stopwatch` (classe `Benchmark`) : un premier timer mesure `UpdateTree()` (catégorie *updateTree*), un second mesure `GetPairs()` (catégorie *broad*) et un troisième mesure la boucle GJK+EPA+résolution (catégorie *narrow*). Les temps sont accumulés chaque frame dans des variables qui calculent moyenne, min et max sur 500 frames (`countToPrint`), ce qui lisse les irrégularités tout en conservant les pics. Les FPS sont déterminé via `Time.deltaTime` (inverse du temps écoulé entre deux frames rendues). Le nombre de paires candidates est également enregistré à chaque frame pour comparer son évolution avec celle du temps de la narrow phase.

6 Conclusion et gestion de projet

J'ai réalisé un moteur physique complet sans `Rigidbody` Unity : broad phase par arbre AABB dynamique, narrow phase GJK+EPA et réponse avec rotation, validés sur un plateau de billes jusqu'à 1 000 objets. La narrow phase est le principal poste de calcul. Parmi les pistes d'amélioration : manifold de contacts (plusieurs points par paire pour stabiliser les empilements), détection continue (éviter le tunneling des billes rapides), et rééquilibrage de l'arbre par rotations.

Difficultés rencontrées :

— **Objets statiques mal filtrés** : au début, les piquets généraient des paires entre eux et la résolution divisait par une masse inverse nulle. Résolu en filtrant dans `CrossChildren()` (au moins un corps dynamique) et en court-circuitant `ResolveCollision()` si `sumInvMass == 0`.

— **Indices de colliders désynchronisés** : `RemoveAtSwapBack()` invalide l'indice du collider déplacé, ce qui cassait `leafOf` et provoquait des paires fantômes. Résolu en mettant à jour `leafOf` du collider déplacé lors du swap.